

Space Risk Model – Satellite Telemetry Analysis

A single self-contained Python script that solves 10 analytical tasks on satellite telemetry (predicting a failure within the next 12 hours) and produces a `submission.csv` answer file.

Table of Contents

- [Data](#)
- [Quick Start](#)
- [Output Format](#)
- [Tasks Solved](#)
- [Implementation Notes](#)
- [Data Integrity Checks](#)
- [Repository Structure](#)
- [Requirements](#)

Data

Input file `space_risk_model.csv` : one row per satellite orbit.

Group	Columns
Identifiers	satellite_id , orbit_number , orbit_type (0 = LEO, 1 = MEO, 2 = GEO)
Target	failure_in_12h (0/1, roughly 2% positive)
Temperatures	solar_panel_temp_mean , battery_temp_mean , computer_temp_mean , transmitter_temp_mean
Battery	battery_voltage_mean , battery_current_mean , battery_voltage_std , battery_current_std
Attitude and thrusters	reaction_wheel_current_mean , reaction_wheel_current_std , attitude_error_mean , thruster_firings_last_24h
Model predictions	model_1_pred ... model_10_pred (probabilities in 0..1)

Expected size: 180,300 rows, 30 satellites. The data itself is not included in this repository.

Quick Start

```
git clone
https://github.com/<your_username>/space-
risk-model.git
cd space-risk-model

python -m venv .venv
source .venv/bin/activate          # Windows:
.venv\Scripts\activate
pip install numpy pandas scikit-learn

# place space_risk_model.csv next to the
script, or pass its path as an argument
python solve_space_risk.py
space_risk_model.csv
```

A full run takes roughly 2–3 minutes; most of the time goes to question 10 (ensemble search). Progress is printed to the console.

Output Format

The script writes two identical files: `submission.csv` and `submission_space.csv`.

```
question_id,answer_1,answer_2,answer_3
```

Unused columns are left empty. Values that contain lists (for example, model numbers) are quoted.

Tasks Solved

#	Task	Result
1	Satellite with the highest mean solar panel temperature for each orbit type	<code>satellite_id</code> for LEO, MEO, GEO
2	Top 5 models by ROC-AUC (ties in predictions broken with <code>rank(method='first')</code>)	model numbers and AUC values
3	Ratio of mean heating rate before a failure (orbit k-3) to mean heating rate in normal operation	<code>round(B / A)</code>
4	Model with the lowest FPR at guaranteed Recall = 1 (threshold = minimum prediction among failures)	model number and FPR
5	Cross-correlation of panel and battery temperature for lags -5 to +5, averaged over satellites	best lag and r
6	Partial correlations via three Pearson coefficients for two variable triples	<code>max(abs(r1), abs(r2))</code>
7	Weighted risk index: weight grid with step 0.01 and all thresholds, maximizing F1	maximum F1

#	Task	Result
8	Correlation between the anomaly fraction of <code>reaction_wheel_current_std</code> (threshold $\mu + 3\sigma$) and failure count across satellites	r
9	Linear regression of battery temperature; ratio of residual std for failures vs. normal orbits	<code>std1 / std0</code>
10	Three-model ensemble with weights and threshold: minimize loss subject to recall and alert-rate constraints (globally and per orbit type)	model triple, threshold, loss

Implementation Notes

Question 2. To make ties in predictions deterministic, values are replaced by ranks via `rank(method='first')` on data sorted by `(satellite_id, orbit_number)`.

Questions 3 and 5. Lags are computed from the orbit number (`orbit_number - k`) rather than the row position. If the data has gaps in orbit numbering, neighboring orbits are not stitched together across the

gap. On gap-free data this is equivalent to

```
groupby().shift(k) .
```

Question 5. The task statement says just "battery", so `battery_temp_mean` is used by default. The column is set by the `Q5_BATTERY_COL` constant at the top of the script. For reference, the script also prints the best lag for battery voltage and current.

Question 7. For each weight set (5,151 combinations), F1 over all unique thresholds is computed in a single vectorized pass over the sorted scores: $F1 = \frac{2 \cdot TP}{(k + P)}$, where k is the number of alerts and P is the number of failures.

Question 10.

- The search runs in two stages: all 120 model triples with weight step 0.05, then refinement with step 0.01 around the best triple.
- Constraints are checked in integer arithmetic (for example, $10 \cdot FN \leq Pos$ instead of $FN/Pos \leq 0.10$) to avoid floating-point rounding errors.
- Only thresholds with an alert rate of at most 25% are feasible, so it is enough to inspect the top part of the sorted ensemble.
- Tie-breaking order: minimum loss L , then minimum threshold t , then the lexicographically smallest model triple.

- The fast implementation was cross-checked against a direct brute-force implementation of the specification on subsamples, with no discrepancies found.

Data Integrity Checks

On load, the script prints the row count, satellite count and failure rate, and compares the size with the expected 180,300 rows. If the CSV is corrupted (NUL bytes, rows with the wrong number of fields), the broken rows are dropped and a warning is printed.

Answers computed on incomplete data may differ from the reference answers, so make sure there are no warnings before submitting.

Repository Structure

```
.
├── solve_space_risk.py    # solves all 10
questions
├── README.md
├── .gitignore            # recommended:
exclude *.csv
└── space_risk_model.csv  # data (not
stored in the repository)
```

Requirements

- Python 3.9+
- numpy
- pandas
- scikit-learn